Project 1 | SIMD advantage

System: M3 Pro (6 P + 6 E cores; 12 physical/logical), 18GB, macOS 15.6.1, Xcode 26.3 / Apple Clang 17.0.0. NEON is 128-bit.

Hypothesis

Regular float32 loops should gain more than float64; larger footprints, strides and ordered reductions should compress SIMD gains.

Method

AXPY, multiply and stencil were tested with scalar, auto and native builds: -O3 -ffp-contract=off; scalar disables loop/SLP vectors, native adds -mcpu=native; no fast-math/LTO. Initialized 128B-aligned arrays, warmup and reference checks precede ~30ms batches. Five shuffled trials; plots use median +/- sample SD; cycles come from real rusage deltas.

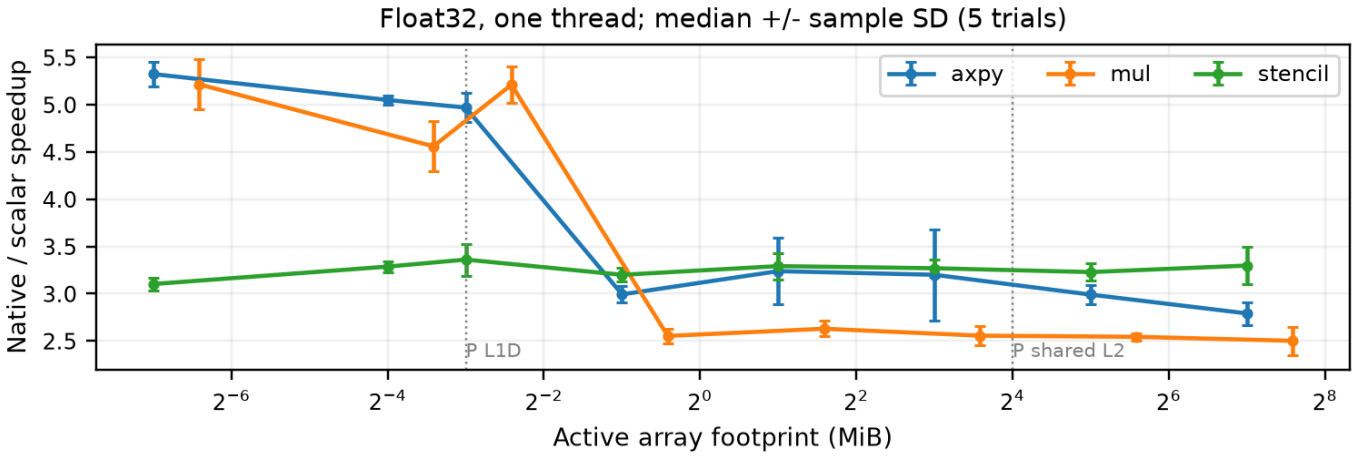


Figure 1. Native float32 speedup across active footprints; gray lines mark P-core L1D (128KiB) and shared L2 (16MiB). Full scalar/auto/native and float64 plots accompany the report.

Native / scalar median speedup	f32, N=1,024	f32, N=16,777,216	f64, N=16,777,216
axpy	5.32x	2.79x	1.46x
mul	5.22x	2.50x	1.32x
stencil	3.10x	3.30x	1.67x

Interpretation

AXPY/multiply gains shrink as movement grows; stencil remains near 3.3x in float32. Lane counts (4 float32 / 2 float64) do not guarantee equal speedup. For large multiply, stride 1 to 8 reduces useful throughput from 106 to 10.2GiB/s and native speedup to ~0.99x. Shifted AXPY addresses cost about 10-47% in tested cases; N/N+1 effects include tails and loop overhead.

STREAM Copy measured 107.1GB/s; 16 verified register FMAs estimate 122.1GFLOP/s. Float32 arithmetic intensities are AXPY 2/12, multiply 1/12, stencil 5/8 FLOP/B. Roofline measured values are 20.0, 9.1 and 49.0GFLOP/s: memory traffic constrains low-intensity loops. AXPY exceeds the Copy-derived line by ~12%, so it is an approximate traffic-dependent reference.

Verification

All 1,200 primary rows and 324 boundary checks passed. Compiler remarks/disassembly verify vector width and scalar tails. The compute-ceiling loop contains the expected 16 vector FMAs.

Negative case

Ordinary dot uses SIMD products but serial additions (~1.00-1.09x); 30 volatile-accumulator controls verify an explicit non-vectorized loop. The incorrect 9-FMA pilot was rejected. Two migrated rows remain included.

Additional required figures: baseline_speedup, baseline_throughput, cycles_per_element, alignment_tail, stride, data_type, roofline (project1/figures/). Detailed numbers and negative-control evidence remain with the project.

Project 2 | Cache and memory

Hypothesis

Random dependent loads should slow beyond local caches; concurrency should initially increase bandwidth, then loaded latency. Page spreading should increase translation pressure, with cache conflicts as a confounder.

Method

Native C equivalents of MLC use prefaulted, warmed seeded chains and real cycle counters. Pages are 16KiB; cache lines are 128B. P-core L1D/shared L2 are 128KiB/16MiB; E-core equivalents are 64KiB/4MiB. No isolated L3 is exposed. The matrix has 355 rows (71 cases x5): sequential/random, 64/256/1024B spacings, 100/0, 0/100, 70/30 and 50/50 read/write, footprint and thread sweeps. Traffic buffers total ~256MiB plus a 64MiB read probe; batches last ~120ms. Useful throughput counts 8B operations, not bus bytes or store persistence.

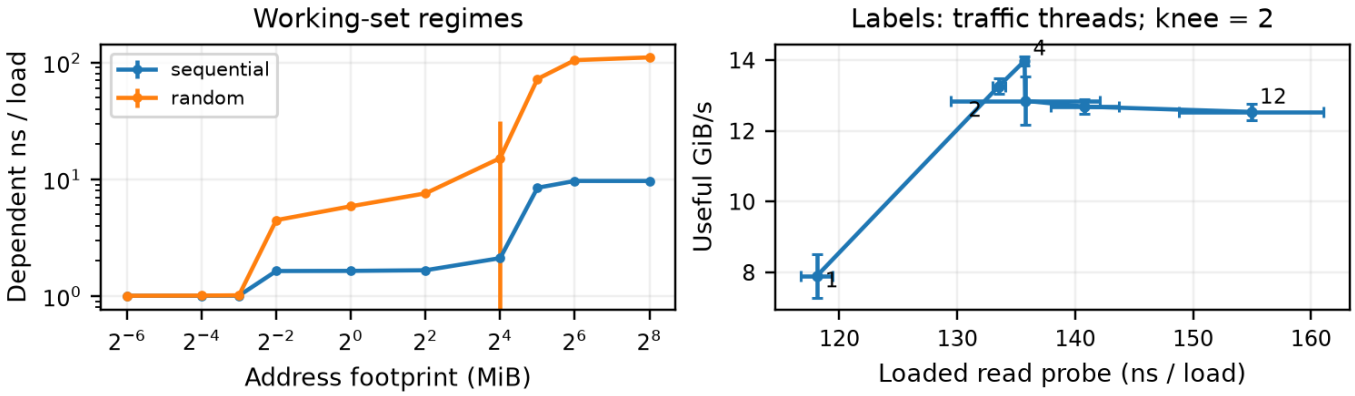


Figure 2. Working-set latency (left); throughput/intensity and loaded-read-latency tradeoff (right). Medians +/- sample SD.

Observed random-load regime	Footprint	Median ns/load	Median cycles/load
P-core-L1-scale	0.0625MiB	1.01	4.02
L2-scale	1MiB	5.87	23.53
Beyond local L2 / possible SLC	32MiB	72.22	290.48
DRAM-scale	256MiB	110.48	442.35

Interpretation

The latency table describes observed regimes, not isolated physical-level constants; sequential prefetching can lower latency. Throughput rises from 7.89GiB/s (1 worker) to 13.26 (2) and 13.98 (4), then falls to 12.53 (12). The >=90%-of-maximum knee is 2 workers. Loaded reads rise from ~118 to 154ns; scheduling and E-core use also matter. Little's Law L = lambda W applies to one request population; stream rate times a different probe's latency is only illustrative. Larger numeric footprints slow with more cache misses. AMAT needs per-access probabilities: the chase-only 66/64 instruction-to-load conversion underpredicts the largest case (~43ns vs 110ns), so it is not a validated multilevel model.

Verification

Cycle uniqueness and read/write references passed. All 42 Instruments captures (3 per case) retain traces/XML and leaf-kernel attribution. Events include L1D load-cache/TLB misses, L2 data-TLB misses, cycles and instructions; misses are normalized per 1,000 sampled instructions. INST_SIMD_ALU also counts scalar FP. Explicit 2MiB superpage allocation failed (invalid argument); no 4KiB/huge-page result is fabricated.

Negative case

Reusing 64KiB (~9.12GiB/s) is a misleading DRAM test versus 256MiB (~7.89GiB/s); loop overhead also prevents claiming physical peak bandwidth. Page-offset balancing and 63/64-load controls preserve the cache-conflict and tiny-cycle anomalies without inventing an architectural cause.

Other plots: working_set_latency, throughput_latency, granularity_latency, pattern_rw_granularity, hardware_counters, numeric_cache_correlation, page_locality, page_offset_control, cache_reuse_pitfall, tiny_cycle_unroll (project2/figures/). All 33 supplementary timings and failed page-support evidence are retained.

Project 3 | SSD performance

Hypothesis

IOPS should plateau and latency rise with concurrency; larger blocks help throughput; buffering may measure RAM.

Method

FIO 3.43/posixaio tests an owned 1GiB disposable file on APPLE SSD AP1024Z (~1TB), Apple Fabric/APFS: 16KiB memory alignment, incompressible buffers, direct=1/F_NOCACHE. No raw device was written; completion without per-I/O fsync does not prove NAND persistence. There are 47 cases x3 shuffled trials (141 performance jobs; 147 directional CSV rows) plus 2 preparation jobs: six block sizes (4-256KiB), sequential/random read/write, required mixes, QD1-256, 2/4/8/12 forked processes and 64/256/1024MiB logical ranges. Main jobs use 1s and byte caps. Plots show medians +/- SD; tails include every trial and min-max.

QD1 pattern	Mean latency (us)	p95 (us)	p99 (us)
randread, 4KiB	78.2	90.6	138.2
randwrite, 4KiB	148.5	179.2	280.6
read, 128KiB	177.2	197.6	242.7
write, 128KiB	40.2	50.9	79.4

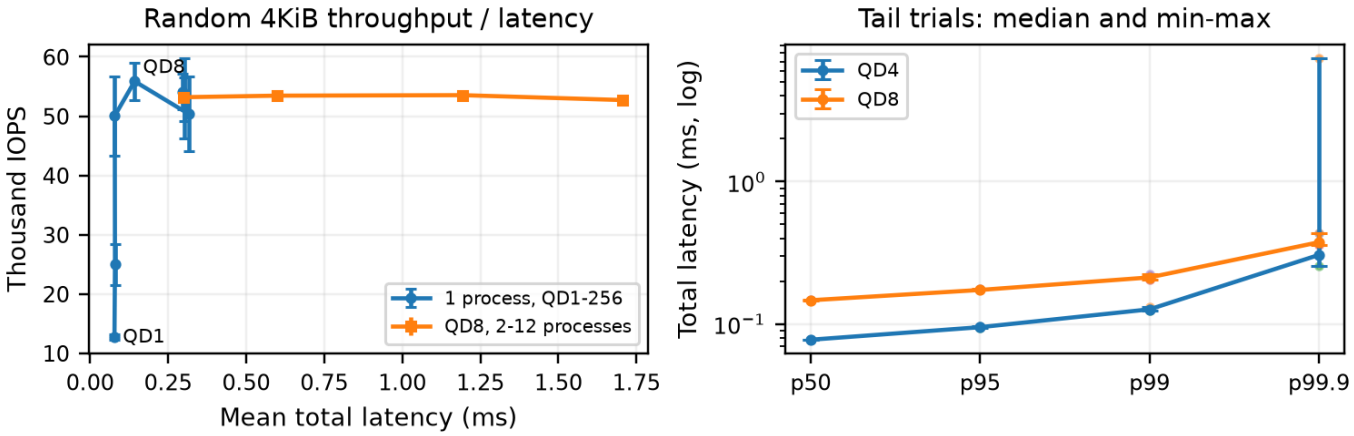


Figure 3. Throughput versus total I/O latency, including forked processes (left); all tail trials with median and min-max (right).

Interpretation

The single-process >=90%-of-maximum knee is QD8 (~55.9k median IOPS). Forked processes reach ~90 in-flight requests yet stay near 53k IOPS as mean latency rises to ~1.71ms. L = IOPS x mean total latency describes the application population, not controller occupancy. Highest individual read rates were 64,952 random IOPS and 3,955MiB/s sequential; no numerical Apple Fabric/PCIe ceiling is inferred. Temperatures were 28-39C; physical APFS LBAs are unknown. Existing-state burst tests do not establish steady state, SLC exhaustion or thermal throttling.

Read-only tail run	p50 (us)	p95 (us)	p99 (us)	p99.9 median / max (us)
tail-mid-q4	77.3	94.7	126.5	305.2 / 7372.8
tail-near_knee-q8	146.4	173.1	212.0	374.8 / 432.1

Verification

Separate 64MiB CRC32C write/read validation passed; no I/O errors. Payload writes were 53.375GiB (<104GiB cap). All six read-only tail jobs completed >=123,209 requests (~123 expected beyond p99.9); all four percentiles and counts are retained. Total/completion latency differ; directional percentiles are not averaged. The ~7.37ms p99.9 outlier remains; scheduling/background I/O or device management are plausible, not proven GC.

Negative case

Requested QD32-256 produces only ~16 average in-flight requests, so requested depth is misleading. Warmed buffered reads strongly inflate apparent SSD IOPS; the direct-I/O comparison exposes the page-cache effect.

Complete plots, FIO configs, JSON+ histograms, temperatures and write ledger: project3/.